GUION DE EXPOSICIÓN

Planificación y ejecución de pruebas de software en un sistema embebido de control de temperatura

CARLOS QUISPE LOPEZ

FIUBA · Testing de Software de Sistemas Embebidos

Guion organizado según las 21 diapositivas de la presentación revisada. Incluye el discurso sugerido, conceptos clave, interpretación de resultados y posibles preguntas del profesor.

Duración orientativa: 15 a 18 minutos. El texto está redactado para decirlo en voz alta; no es necesario memorizarlo literalmente.

1. Contexto y arquitectura

Diapositivas 1 a 3 · aproximadamente 2 minutos

Diapositiva 1. Portada

Buenas tardes, profesor y compañeros. Mi nombre es Carlos Quispe Lopez y voy a presentar mi trabajo de planificación y ejecución de pruebas de software. El proyecto seleccionado es un sistema embebido de control de temperatura basado en el STM32F446RE. Apliqué dos enfoques: pruebas de transición de estados, o STT, sobre la máquina de estados del sensor, y pruebas unitarias con Ceedling, Unity y CMock sobre el controlador RTC DS3231.

Diapositiva 2. Contexto del proyecto

El sistema permite monitorear y controlar una temperatura utilizando un sensor analógico MCP9700A. El STM32 adquiere la medida, la compara con los parámetros configurados y decide si debe activar o desactivar una salida. Para evitar conmutaciones repetidas cerca del valor objetivo, utiliza histéresis. Desde testing, buscamos verificar que las decisiones sean correctas ante lecturas normales y que la respuesta sea segura si falla la adquisición.

Clave para explicar: Histéresis: utiliza umbrales distintos de activación y desactivación.

Diapositiva 3. Constitución del sistema

La arquitectura está centrada en el STM32F446RE. El MCP9700A entrega una señal analógica al ADC. El firmware procesa esa señal y ejecuta el control en sensor_fsm.c, que gobierna la salida PA5. El sistema además dispone de UART para configuración y diagnóstico, LCD para visualización y un RTC DS3231 conectado por I2C para fecha y hora. Seleccioné sensor_fsm.c para STT y ds3231.c para las pruebas unitarias.

Clave para explicar: Señala los bloques del diagrama mientras explicas el flujo: medir, decidir, actuar e informar.

2. Planificación del testing

Diapositivas 4 a 6 · aproximadamente 2 minutos

Diapositiva 4. Características de calidad

Para desarrollar el plan de pruebas prioricé adecuación funcional con 35 %, fiabilidad con 30 % y seguridad funcional básica con 20 %. Estas características son las más importantes porque el sistema debe medir, decidir y desactivar la salida si recibe una lectura inválida. También consideré mantenibilidad con 10 % y eficiencia temporal con 5 %.

Clave para explicar: Estos pesos son decisiones de planificación, no resultados ni cobertura medida.

Diapositiva 5. Módulos y criticidad

Clasifiqué los módulos según el impacto de sus fallas. sensor_fsm.c, mcp9700a.c y adc_drv.c presentan criticidad alta por intervenir directamente en la lectura y el control. El RTC, I2C y UART tienen criticidad media dentro del alcance definido. Esta selección me permitió concentrar las pruebas donde existe mayor efecto sobre el comportamiento funcional.

Diapositiva 6. Métodos y exclusiones

Para la máquina de estados utilicé STT, identificando transiciones y construyendo recorridos de prueba. Para DS3231 apliqué pruebas unitarias con Ceedling, Unity y CMock. Las dependencias de hardware se simularon, por lo que el estudio no mide precisión física del sensor, comportamiento eléctrico ni funcionamiento completo extremo a extremo sobre la placa STM32.

Clave para explicar: La simulación permite controlar errores difíciles de provocar físicamente, pero tiene límites.

3. Diseño, ejecución y resultados STT

Diapositivas 7 a 13 · aproximadamente 5 a 6 minutos

Diapositiva 7. Máquina de estados del sensor

En el archivo sensor_fsm.c identifiqué cinco estados: IDLE, READ, CONTROL, OUTPUT y ERROR. IDLE espera el período de 500 ms; READ adquiere y valida la temperatura. Con una lectura válida, CONTROL aplica la histéresis y OUTPUT actualiza las salidas antes de regresar a IDLE. Si la lectura es inválida, la máquina pasa a ERROR, desactiva la salida y retorna a IDLE.

Clave para explicar: Sigue con el puntero un recorrido normal y otro de error.

Diapositiva 8. Tabla de transiciones

A partir de la máquina construí una tabla STT. Para cada transición registré el estado inicial, la condición o evento y el estado siguiente. Identifiqué nueve transiciones en este modelo. T1 verifica que el programa permanezca en IDLE antes de cumplir 500 ms; T2 verifica el cambio a READ al cumplirse ese tiempo. T4, T5 y T6 corresponden a diferentes condiciones de error. El criterio fue ejecutar al menos una vez cada transición identificada.

Clave para explicar: La tabla convierte el comportamiento del sistema en condiciones verificables.

Diapositiva 9. Cuatro recorridos de prueba

Diseñé cuatro recorridos. R1 verifica el funcionamiento normal usando 32 °C, setpoint de 30 °C e histéresis de 1 °C: se espera activar PA5. R2 introduce un fallo ADC. R3 simula una lectura fuera de rango. R4 introduce -5 °C y comprueba que la aplicación la rechace. Con ellos cubro la ruta funcional y distintas variantes de error.

Diapositiva 10. Implementación del caso STT

Utilicé el código productivo sensor_fsm.c, junto a un entorno de pruebas escrito en C. Ese entorno simula HAL_GetTick(), el sensor MCP9700A, el GPIO PA5 y otras dependencias. Invoca repetidamente sensorFsmUpdate() y verifica las respuestas observables. El proyecto se compiló con GCC y se ejecutó desde Windows con un archivo BAT.

Clave para explicar: Aclara que no compilaste todo el proyecto STM32 en la PC; aislaste la MEF y reemplazaste sus dependencias.

Diapositiva 11. Evidencia de ejecución STT

En la terminal se observan mensajes PASS para las transiciones T1 a T9. La ejecución incluyó cuatro recorridos y nueve verificaciones aprobadas. Esto demuestra que, para las entradas ensayadas y en el entorno simulado, la máquina produjo las respuestas esperadas.

Diapositiva 12. Resultados observados por recorrido

En R1, una lectura válida de 32 °C provocó la activación de PA5 y el retorno a IDLE. En R2, el error ADC llevó a ERROR y desactivó la salida. En R3, el error de rango fue reconocido. En R4, una lectura negativa se rechazó. Los cuatro recorridos aprobaron; se comprobaron tanto el funcionamiento normal como la reacción frente a errores.

Diapositiva 13. Análisis de cobertura STT

Las nueve transiciones identificadas en la tabla STT fueron ejecutadas al menos una vez, alcanzando el 100 % de cobertura de transiciones de ese modelo. Esto no equivale al 100 % de líneas o ramas del firmware ni demuestra por sí solo el comportamiento sobre el STM32 físico. La validación de hardware queda como etapa posterior.

Clave para explicar: No confundas cobertura de transiciones de sensor_fsm.c con cobertura de código del ds3231.c.

4. Pruebas unitarias y cobertura del DS3231

Diapositivas 14 a 20 · aproximadamente 4 a 5 minutos

Diapositiva 14. Módulo bajo prueba y herramientas

Seleccioné el driver DS3231 porque gestiona funciones de fecha y hora. Creé test_ds3231.c, donde definí entradas, expectativas y resultados esperados. Ceedling organiza y automatiza la ejecución; Unity comprueba resultados mediante aserciones; CMock simula el bus I2C y verifica llamadas realizadas por el driver.

Diapositiva 15. Casos iniciales del RTC

Inicialmente preparé nueve casos de prueba: conversiones de formato horario, lectura y escritura BCD, detección de dispositivo ausente y manejo de punteros nulos. También probé una secuencia WriteTime seguida de ReadTime con I2C simulado; esa secuencia no constituye integración real con i2c_drv.c.

Clave para explicar: No es necesario leer toda la tabla: resume los grupos de comportamiento que cubren las pruebas.

Diapositiva 16. Ampliación a trece pruebas

La primera medición de cobertura mostró código sin ejecutar. Añadí cuatro casos: inicialización correcta, escritura de fecha, lectura de temperatura interna del RTC y fallo I2C en una operación de lectura. Con ello pasé de nueve a trece pruebas aprobadas y amplié las rutas de código evaluadas.

Diapositiva 17. Ejemplo unitario: conversión 12 h a 24 h

Construyo el registro 0x63, que representa las 3 PM en formato de 12 horas. Después llamo a DS3231_ApplyHourFormat() para convertirlo a 24 horas. Unity comprueba que el valor obtenido sea 0x15, es decir, 15 horas en BCD, y que el bit de formato 12 horas quede desactivado. Es una prueba de lógica que no requiere comunicación I2C.

Diapositiva 18. Ejemplo unitario: RTC no disponible

Para simular un dispositivo ausente, configuro i2cDeviceReady_ExpectAndReturn(DS3231_ADDR, false). CMock devuelve false cuando el driver consulta el I2C. Después ejecuto DS3231_Init() y Unity comprueba con TEST_ASSERT_FALSE que DS3231_IsAvailable() informa que el RTC no está disponible. La ventaja es poder reproducir una falla sin desconectar físicamente el dispositivo.

Diapositiva 19. Ejecución automatizada con Ceedling

Ceedling compiló el driver y los archivos de prueba, generó mock_i2c_drv.c y ejecutó los casos con Unity. El resultado fue de trece pruebas ejecutadas, trece aprobadas y cero fallidas. Sin embargo, aprobar una prueba no significa haber ejecutado todas las líneas o alternativas del software; por eso medí cobertura.

Diapositiva 20. Cobertura GCOVR

Mediante GCOV y GCOVR medí la cobertura del archivo ds3231.c. Obtuve 72,3 % de líneas: 99 de 137; 100 % de funciones: 13 de 13; y 52,7 % de ramas: 39 de 74. Esto significa que todas las funciones se ejecutaron alguna vez, pero todavía existen decisiones y rutas alternativas no recorridas. Los porcentajes permiten orientar nuevos casos de prueba.

Clave para explicar: La cobertura de funciones es una medida de ejecución, no una garantía de corrección de cada función.

Resumen de resultados que debes recordar

Indicador	Resultado	Interpretación
STT	9/9 transiciones; 4 recorridos	100 % de las transiciones del modelo identificado
Ceedling	13/13 pruebas aprobadas	Todos los escenarios unitarios definidos aprobaron
GCOVR - líneas	72,3 % (99/137)	Quedan líneas sin ejecutar
GCOVR - funciones	100 % (13/13)	Todas se ejecutaron al menos una vez
GCOVR - ramas	52,7 % (39/74)	Quedan alternativas de decisión pendientes

5. Conclusión

Diapositiva 21 · aproximadamente 1 minuto

Diapositiva 21. Conclusiones y trabajo futuro

> Como conclusión, apliqué STT a la máquina de estados del control de temperatura, ejecutando cuatro recorridos que verificaron nueve transiciones identificadas. También realicé trece pruebas automatizadas del controlador DS3231 con Ceedling, Unity y CMock, todas aprobadas. La medición GCOVR mostró que aún existen ramas sin cubrir, por lo que como trabajo futuro propongo ampliar escenarios de error, realizar integración I2C real y validar el sistema sobre STM32. Muchas gracias.

6. Preguntas probables del profesor

Respuestas breves para la sustentación

¿Cuál es la diferencia entre STT y Ceedling?

STT es un método de diseño de pruebas basado en estados y transiciones. Ceedling es una herramienta que automatiza compilación y ejecución de pruebas C.

¿Por qué seleccionaste sensor_fsm.c?

Porque define las decisiones de control y las rutas de manejo de fallos; es crítico para el comportamiento de la salida.

¿Qué diferencia existe entre Unity y CMock?

Unity verifica resultados mediante aserciones. CMock genera sustitutos de dependencias y permite establecer expectativas de llamada y respuestas.

¿Por qué hablas de 100 % STT pero 52,7 % de ramas?

Son dos métricas y módulos diferentes: STT mide transiciones de sensor_fsm.c y GCOVR mide ramas ejecutadas de ds3231.c.

¿Por qué 100 % de funciones no implica cobertura completa?

Porque ejecutar una función al menos una vez no obliga a recorrer cada alternativa interna.

¿La secuencia WriteTime y ReadTime es integración real?

No. Utiliza el driver real, pero I2C sigue reemplazado por CMock. La integración real con i2c_drv.c queda pendiente.

¿Se realizaron las pruebas sobre la placa física?

No; se ejecutaron en PC con dependencias simuladas. La validación eléctrica, temporal y física sobre STM32 está fuera del alcance realizado.

Consejos finales para exponer

1. Señala el flujo del diagrama en lugar de leer cada bloque. 2. En STT distingue el diseño, el estímulo y el resultado observado. 3. En las pruebas unitarias explica quién escribe el test, quién genera el mock y quién verifica el resultado. 4. Al hablar de porcentajes menciona siempre qué se está midiendo. 5. No presentes las pruebas simuladas como una validación física completa del STM32.